НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Дисциплина: Системы искуственного интелекта

Модуль 1.
Лабораторная работа №1
И
Лабораторная работа №2

Выполнил студент:

Немыкин Ярослав Алексеевич

Группа P3322

Преподаватель:

Королёва Юлия Александровна

г. Санкт-Петербург, 2026

Table of contents (Оглавление)

АННОТАЦИЯ 3

ВВЕДЕНИЕ 4

БАЗА ЗНАНИЙ В PROLOG (ЛР1, ЧАСТЬ 1) 5

Выбранный домен и словарь сущностей 5
Состав базы знаний 5
Структура исходного файла 6
Реализованные правила 7
Тестовые запросы и ответы интерпретатора 8

ОНТОЛОГИЯ В PROTÉGÉ (ЛР1, ЧАСТЬ 2) 10

Перевод Prolog → OWL 10
Классы и иерархия 10
ObjectProperty и DataProperty 11
Кардинальности и функциональность 12
Несовместимости и ограничения 12
Проверка reasoner и ограничения среды 12
Компетентностные вопросы и ожидаемые ответы 14

DSS НА БЗ (ЛР2) — АРХИТЕКТУРА И РЕАЛИЗАЦИЯ 16

Формат входа 16
Конвейер 16
Примеры сценариев 17
Обработка нештатных случаев 17

ТЕСТИРОВАНИЕ И ОТЛАДКА 19

Тест-кейсы для Prolog-запросов 19
Тест-кейсы для DL Query (Protégé) 19
Тесты DSS 19

ОЦЕНКА И ИНТЕРПРЕТАЦИЯ 21

Сравнение реализаций: Prolog vs OWL/Protégé 21

ЗАКЛЮЧЕНИЕ 22

ПРИЛОЖЕНИЕ 23

Аннотация

В рамках модуля разработана система поддержки принятия решений (СППР / DSS) для предметной области настольных ролевых игр (НРИ), включающая игроков, персонажей, классы, расы, противников и места рождения. Работа выполнена в три этапа, демонстрирующих полный цикл формализации и использования знаний.

На первом этапе предметная область описана в виде фактов и правил логического вывода на языке Prolog, что позволяет получать новые сведения посредством унификации и резолюции. На втором этапе основные сущности и отношения перенесены в OWL-онтологию, созданную в среде Protégé. В онтологии заданы классы, иерархия понятий, объектные и дататип-свойства, а также ограничения функциональности отношений, что делает структуру предметной области явной и пригодной для машинного анализа.

На завершающем этапе реализован программный конвейер СППР на языке Python, обеспечивающий интерфейс на естественном русском языке. Система выполняет распознавание намерений пользователя, извлечение именованных сущностей (с учетом морфологии и синонимов), динамическое формирование запросов к базе знаний Prolog (через библиотеку pyswip) и интеллектуальную агрегацию результатов. Особое внимание уделено обработке контекстных параметров (например, текущего игрового года) и нештатных ситуаций (например, фильтрация персонажей, которые еще не родились).

Получившаяся связка «OWL-онтология (структурная схема) → Prolog (правиловый вывод) → Python-агент (парсинг и диалог)» демонстрирует создание прозрачной, расширяемой и проверяемой системы, основанной на знаниях, готовой к интеграции в реальные игровые или аналитические процессы.

Введение

Цель модуля — освоить полный цикл разработки интеллектуальных систем: от формализации предметной области и логического моделирования знаний до создания работающей системы поддержки принятия решений (DSS). В ходе работы решаются задачи выделения сущностей предметной области, формализации фактов и отношений, построения правил вывода, переноса полученной модели в онтологическое представление, а также разработки программного агента, способного обрабатывать запросы пользователя на естественном русском языке.

В работе рассматриваются взаимодополняющие подходы к представлению и использованию знаний. Prolog позволяет компактно описывать факты и правила и получать ответы на запросы за счёт унификации и логического вывода (конфликты персонажей, возраст, напряжение между игроками). OWL в среде Protégé используется для фиксации классов, индивидов, свойств и ограничений, благодаря чему структура предметной области становится явной и пригодной для машинного анализа. Для практического применения этих знаний разрабатывается DSS-агент на языке Python, который связывает лингвистический интерфейс с логическим ядром: распознаёт намерения пользователя, извлекает именованные сущности с учётом морфологии и динамически формирует запросы к базе знаний через библиотеку pyswip.

Практическая ценность работы заключается в создании прозрачной и проверяемой системы, где каждый ответ пользователя обоснован конкретным правилом или фактом. Особое внимание уделяется контекстной обработке данных: система учитывает текущий игровой год, корректно исключает из выборки персонажей, которые ещё не родились, и комбинирует несколько критериев поиска (например, расу и класс одновременно). Связка «OWL-онтология → Prolog-база знаний → Python-агент» демонстрирует переход от теоретического моделирования к прикладному интеллектуальному инструменту, пригодному для использования в игровой аналитике и других предметных областях, требующих объяснимого принятия решений.

База знаний в Prolog (ЛР1, Часть 1)

Выбранный домен и словарь сущностей

В качестве предметной области выбрана игровая среда. В исходном Prolog-файле выделены игроки, персонажи, противники, классы, расы и деревни. Дополнительные отношения описывают выбор персонажа игроком, класс персонажа, расу, место и год рождения, а также враждебность между расами.

Сущность / отношение	**Арность**	**Назначение**
player	1	игрок
char	1	персонаж
enemy	1	противник
class	1	игровой класс
magical_class	1	магический класс
race	1	раса
village	1	деревня
playing	2	игрок принимает роль персонажа
char_info	2	персонаж относится к классу
hostile_race	2	враждебность между расами
born_race	2	раса персонажа
birth_info	3	год и место рождения

Таблица 1 – словарь сущностей

Состав базы знаний

Файл first.pl содержит 23 факта с одним аргументом и 22 факта с двумя или более аргументами. В базе присутствуют четыре игрока, четыре персонажа, два противника, четыре класса, два из которых магические, четыре расы и три деревни. Отношения представлены привязками игроков к персонажам, классами, расами, враждебностью рас и сведениями о рождении.

В разделе правил реализованы семь правил: knows_history/2, can_inspire/1, will_conflict/2, can_tank/1, can_suppressed/2, party_tension/2 и age/3. Они используют композицию условий, конъюнкцию, дизъюнкцию, отрицание неравенства, а также арифметическое вычисление.

Структура исходного файла

- Факты об игроках, персонажах и противниках.
- Факты о классах и магических классах.
- Факты о расах и враждебности рас.
- Связи персонажей с расами и сведения о рождении.
- Правила логического вывода.

Фрагмент исходной базы знаний:

```
% Ассистент в ведении партий нри
% Игроки
player(ivan).
player(oleg).
player(artem).
player(vanya).

% Персонажи
char(torvald).
char(illidan).
char(shmig).
char(senshi).
% злодеи
enemy(buga).
enemy(vuga).

% Привязки персонажей к игрокам
playing(ivan, torvald).
playing(oleg, shmig).
playing(artem, illidan).
playing(vanya, senshi).


% Классы
class(rogue).
class(bard).
class(mage).
class(warrior).
% Магические классы
magical_class(mage).
magical_class(bard).
```

Рисунок 1 — Фрагмент фактов из first.pl.

Реализованные правила

```
% Воин может танковать
can_tank(Char) :-
    char_info(Char, warrior).

% Персонаж может эмоцианально подавить другого, если разница в возрасте
% больше на 20 лет
can_suppressed(Char1, Char2) :-
    (   char(Char1); enemy(Char1)),
    (   char(Char2); enemy(Char2)),
    Char1 \= Char2,
    birth_info(Char1, Year1, _),
    birth_info(Char2, Year2, _),
    Difference is Year2 - Year1,
    Difference > 20.

% Напряжение между игроками есть, если их персонажи конфликтуют
party_tension(Player1, Player2) :-
    playing(Player1, Char1),
    playing(Player2, Char2),
    Player1 \= Player2,
    will_conflict(Char1, Char2).

% Сколько лет персонажу сейчас
age(Char, Age, Now) :-
    birth_info(Char, Year, _),
    Age is Now - Year.
```

Рисунок 2 — Примеры правил.

Наиболее содержательным является правило will_conflict/2. Оно объединяет несколько альтернатив: конфликт возможен из-за враждебности рас либо из-за участия противника в игровой связи. В условии используются дизъюнкции (;), конъюнкции через последовательность целей и ограничение Char1 \= Char2.

```
% Персонажи будут конфликтовать, если враждебной рассы друг к другу или
% враги
will_conflict(Char1, Char2) :-
    (   char(Char1); enemy(Char1)),
    (   char(Char2); enemy(Char2)),
    Char1 \= Char2,
    born_race(Char1, Race1),
    born_race(Char2, Race2),
    ( hostile_race(Race1, Race2);
    (   enemy(Char1), playing(_, Char2));
    (   enemy(Char2),  playing(_, Char1))
    ).
```

Рисунок 3 — Правила конфликтов.

Тестовые запросы и ответы интерпретатора

Тип	**Запрос**	**Ответ / интерпретация**
Простой поиск факта	?- player(ivan).	true.
Поиск персонажа	?- char(torvald).	true.
Конъюнкция	?- playing(P, C), char_info(C, mage).	P = artem, C = illidan.
Дизъюнкция	?- can_tank(C) ; can_inspire(C).	C = senshi ; C = torvald ; C = illidan ; C = vuga.
Отрицание	?- can_suppressed(C1, C2), \+ will_conflict(C1, C2).	C1 = illidan, C2 = torvald ; C1 = illidan, C2 = shmig ; C1 = senshi, C2 = torvald
Правило истории	?- will_conflict(torvald, Who).	Who = shmig ; Who = buga ; Who = vuga
Правило напряжённости	?- party_tension(P1, P2).	P1 = ivan, P2 = oleg ; P1 = oleg, P2 = ivan ; P1 = oleg, P2 = vanya ; P1 = artem, P2 = vanya ; P1 = vanya, P2 = oleg ; P1 = vanya,

		P2 = artem
Арифметическое правило	?- age(illidan, Age, 2024).	Age = 1024.
Возрастное условие	?- can_suppressed(shmig, torvald).	true.

Таблица 2 – запросы и ответы от интерпретатора

Приведённые ответы отражают логику исходного файла. Для запросов, где Prolog может возвращать несколько решений, фактический порядок вариантов определяется порядком фактов и ветвлений в программе.

Онтология в Protégé (ЛР1, Часть 2)

Перевод Prolog → OWL

В файле first-ontology.rdf исходные понятия представлены средствами OWL/RDF. Выделены классы Character, Enemy, Player, Race, Village, Class и MagicClass. Класс Enemy задан как подкласс Character, а MagicClass — как подкласс Class. Индивиды соответствуют персонажам, игрокам, расам, деревням и классам из Prolog. Сам файл был создан при помощи интерфейса в программе Protégé

Prolog	**OWL элемент**	**Комментарий**
char(X)	Character	Класс персонажей.
enemy(X)	Enemy ⊑ Character	Противник моделируется как подкласс персонажа.
player(X)	Player	Класс игроков.
race(X)	Race	Класс рас.
village(X)	Village	Класс деревень.
playing(P,C)	playing (ObjectProperty)	Player → Character.
char_info(C,Class)	hasClass (ObjectProperty)	Character → Class.
born_race(C,R)	bornRace (ObjectProperty)	Character → Race.
birth_info(C,Year,V)	birthYear + birthPlace	Разделение тернарного факта на два свойства.
hostile_race(R1,R2)	hostileRace (ObjectProperty)	Race → Race.

Таблица 3 — Мини-матрица трассируемости Prolog ↔ OWL.

Классы и иерархия

В текущем RDF-файле зафиксирована компактная иерархия. Enemy является подклассом Character, а MagicClass — подклассом Class. Остальные классы представлены на одном уровне. Это соответствует схеме предметной области, реализованной в исходной версии онтологии.

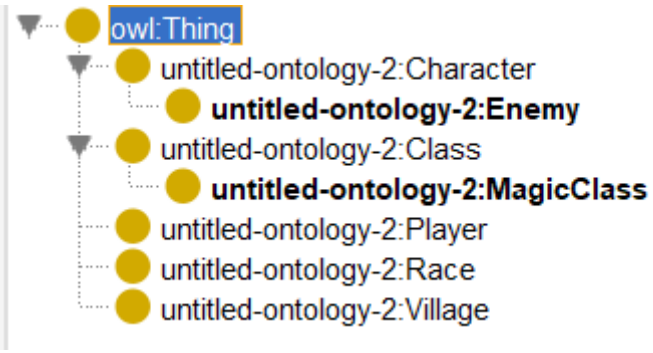


Рисунок 4 — Иерархия классов.

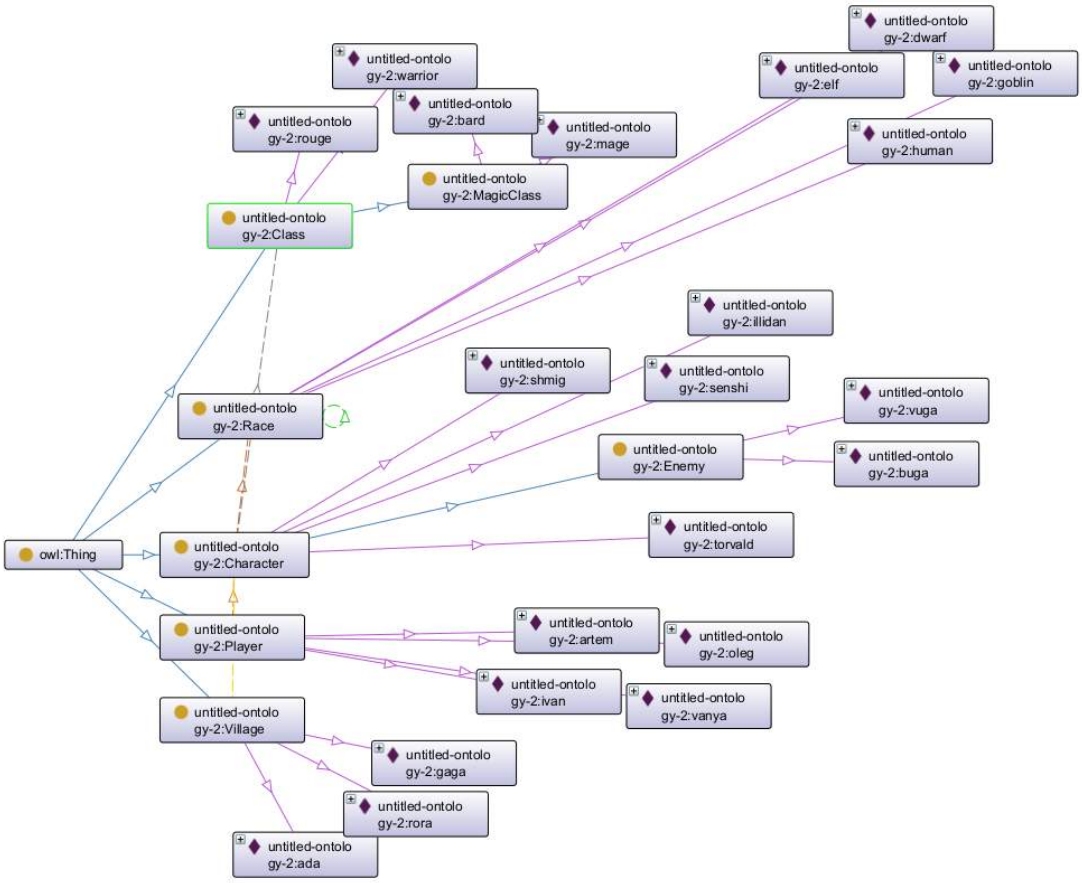


Рисунок 5 – Граф онтологии.

ObjectProperty и DataProperty

В онтологии определены следующие объектные свойства: birthPlace, bornRace, hasClass, hostileRace и playing. Для них заданы домены и диапазоны. Например, birthPlace имеет домен Character и диапазон Village, bornRace — Character и Race, hasClass — Character и Class, playing — Player и Character.

Дататип-свойство birthYear имеет домен Character и диапазон xsd:integer. Тем самым год рождения представлен числовым значением, а не отдельным объектом.

Кардинальности и функциональность

В работе несколько объектных свойств объявлены функциональными (FunctionalProperty): birthPlace, bornRace, hasClass и hostileRace. Свойство playing дополнительно объявлено InverseFunctionalProperty. Это означает, что модель ограничивает число значений соответствующих отношений и задаёт семантику уникальности. Ограничения выражены через функциональность свойств.

Несовместимости и ограничения

В работе явные конструкции owl:disjointWith и числовое ограничение вида Age ≥ 0 отсутствуют. Так как подобных полей не создавалось при проектировании.

Проверка reasoner и ограничения среды

При использовании HermiT проблемы выявлены не были.

Плагины и дополнительные расширения Protégé не удалось подключить. Поэтому полноценное выполнение SPARQL-запросов и SWRL-правил в рамках работы не удалось. Практическая часть в Protégé ограничилась построением онтологической схемы, описанием свойств и размещением индивидов. Однако, удалось составить DL Query запросы.

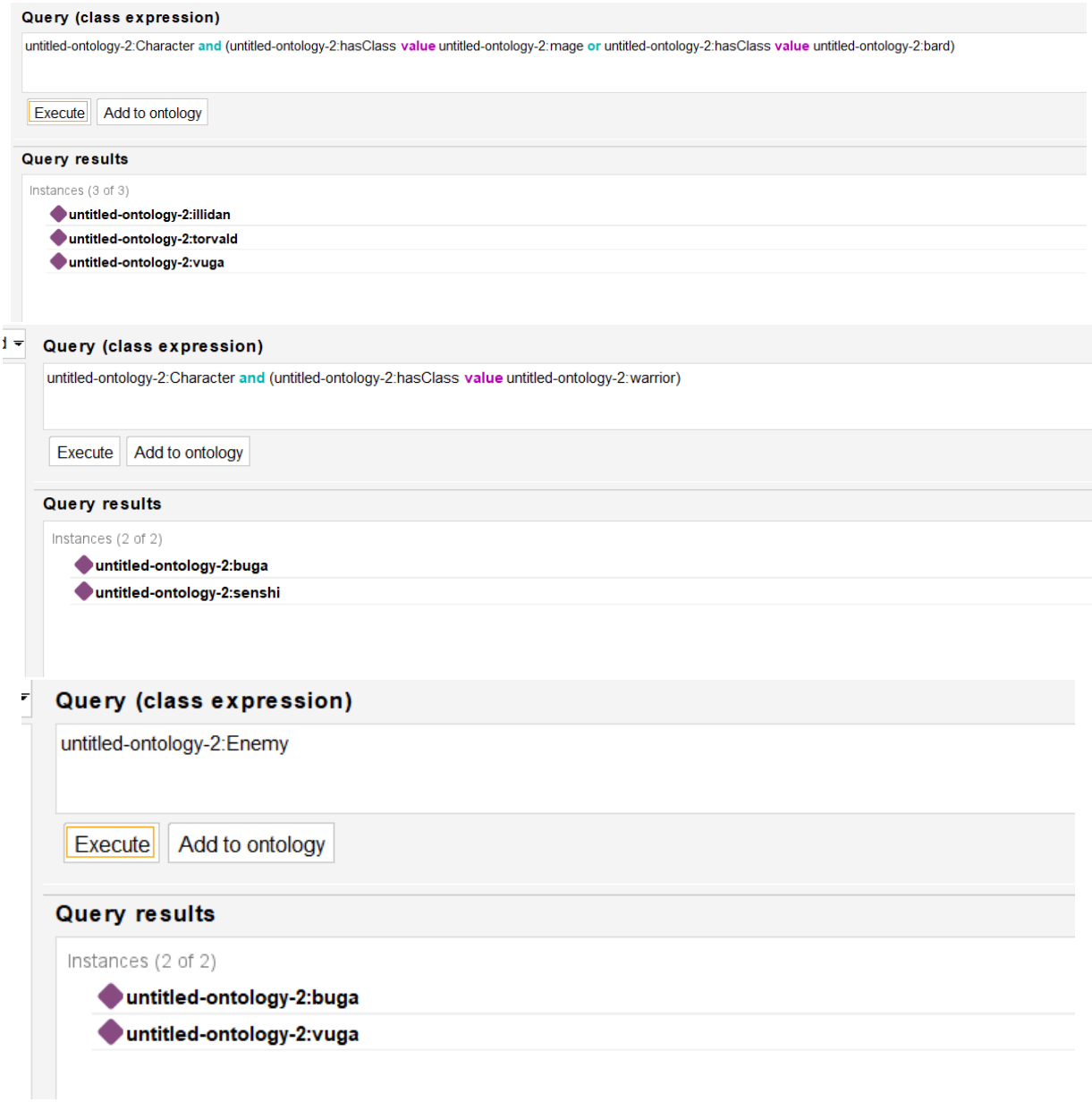


Рисунок 6 – работа DL Qery

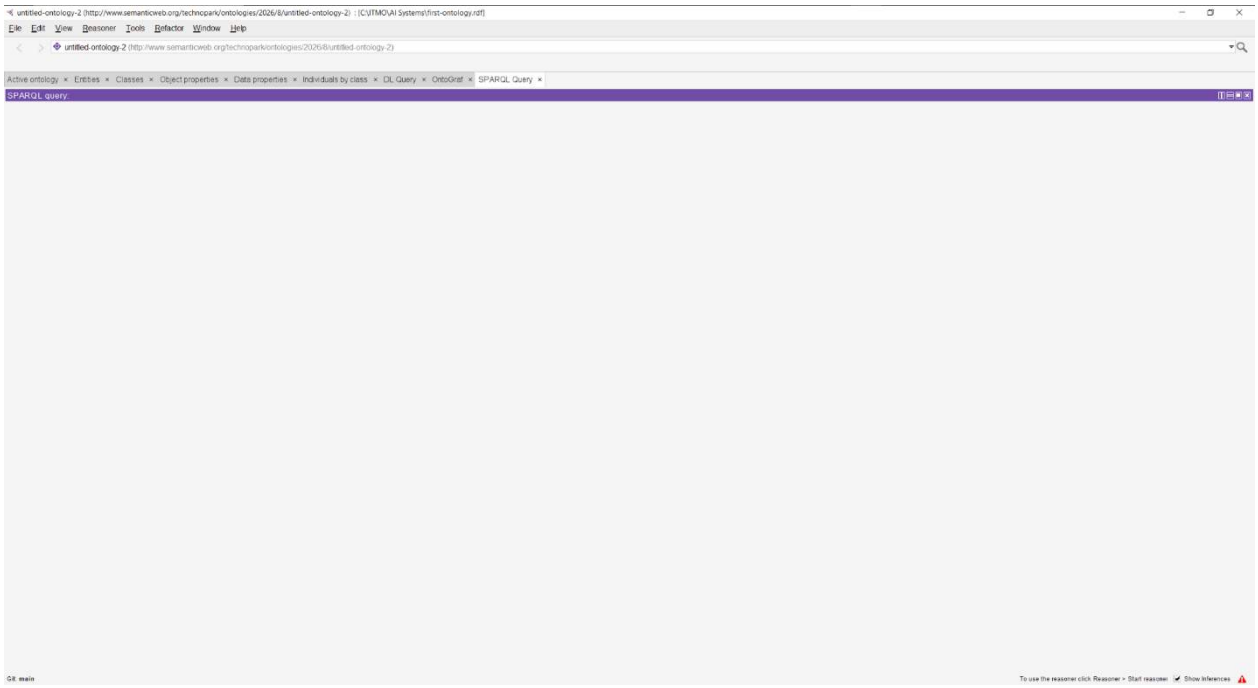


Рисунок 7 – ошибки SPARQL.

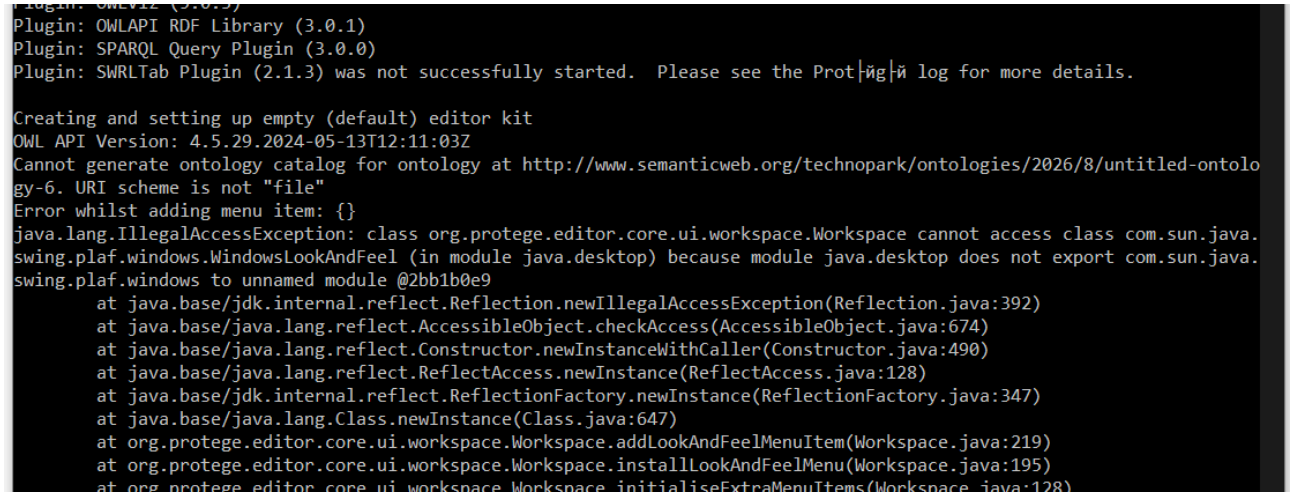


Рисунок 8 – ошибки SWRL.

Компетентностные вопросы и ожидаемые ответы

Поскольку SPARQL и SWRL в текущей среде не использовались, ниже приведены не результаты выполнения запросов, а формулировки компетентностных вопросов, которые проверяют содержание созданной модели. Они могут быть реализованы в Protégé при подключении соответствующих средств.

Компетентностный вопрос	**Ожидаемый ответ по данным RDF**
Какие игроки представлены в онтологии?	ivan, oleg, artem, vanya.
Какими персонажами управляют игроки?	ivan→torvald, oleg→shmig, artem→illidan, vanya→senshi.
Какой класс имеет illidan?	mage.

Какие персонажи родились в деревне ada?	torvald, shmig, senshi.
Какие расы связаны отношением hostileRace?	goblin↔dwarf, goblin↔human, elf↔dwarf.

DSS на БЗ (ЛР2) — архитектура и реализация

Формат входа

На вход системе подаётся **строка на русском языке** в свободной форме, содержащая вопрос о предметной области. Строка нормализуется (приведение к нижнему регистру, замена ё → е, удаление лишних пробелов) и далее обрабатывается конвейером.

Примеры корректных входов:

№	Входная строка
1	Кто может танковать?
2	Могут ли конфликтовать Сенши и Шмыг?
3	Родился ли Торвальд в Аде в 1899 году?
4	Кто из эльфов умеет вдохновлять?
5	Конфликтуют ли Иван и Олег?

Конвейер

Ниже приведён псевдокод основного конвейера, реализованного в process_message:

```
text ← normalize(message)
если text ∈ {помощь, примеры, выход} → выполнить служебную команду
intent ← detect_intent(text)              // поиск самой длинной
// совпавшей фразы из INTENTS
entities ← extract_entities(text)         // слоты:
//   characters, players,
//   classes, races, cities, year
если intent = None → «Не удалось определить тип вопроса»
handler ← handlers[intent]                // маршрутизация к обработчику
answers ← handler(entities)               // запрос к Prolog + фильтрация
вывести answers пользователю
```

Внутри обработчиков: формируется текстовый запрос вида will_conflict(A, B), birth_info(C, Y, V), age(X, Age, 2026) и т.п.; результат получается через pyswip.Prolog.query.

Примеры сценариев

№	Вход	Запрос к Prolog	Ответ системы	Обоснование
1	Кто может танковать?	can_tank(C)	Танковать могут: Сенши, Торвальд.	Прямое правило can_tank/1 из БЗ.
2	Могут ли конфликтовать Сенши и Шмиг?	will_conflict(senshi, shmig)	Да, Сенши и Шмиг могут конфликтовать.	Срабатывает дизъюнкция: враждебность рас + участие противника.
3	Родился ли Торвальд в Аде в 1899 году?	birth_info(torvald, Y, V) с фильтрами	Да, Торвальд родился в Аде в 1899 году.	Сравнение с извлечёнными слотами year=1899, city=ada.
4	Кто из эльфов умеет вдохновлять?	can_inspire(C), фильтр по race elf	Вдохновлять могут: Иллидан.	Комбинация правила и фильтра по слоту races.
5	Конфликтуют ли Иван и Олег?	party_tension(ivan, oleg)	Между игроками Иван и Олег возможно напряжение.	Слоты players, маршрут tension.
6	Найди всех гоблинов	born_race(C, goblin)	Буга — гоблин. Вуга — гоблин.	Слот races, маршрут race_search.
7	Сколько лет Иллидану? (при YEAR=2026)	age(illidan, Age, 2026)	Иллидан: примерно 1026 лет.	Арифметическое правило age/3.

Обработка нештатных случаев

Ситуация	Обработка в коде
Персонаж ещё не родился (birth_year > YEAR)	В answer_birth, answer_age, answer_conflict выводится сообщение «{Имя} ещё не родился», запись исключается из результатов.
Указано больше двух персонажей в парном запросе	answer_conflict возвращает подсказку уточнить пару.
Не удалось определить intent	Печатается «Не удалось определить тип вопроса. Введите «помощь»…».

Сущность не найдена в словаре	Слот остаётся пустым, обработчик работает по общей выборке.
Пустой результат от Prolog	Возвращается «Подходящих … не найдено».
Конфликт признаков (раса + класс, но ни один персонаж не подходит)	Фильтр apply_character_filters последовательно исключает кандидатов.

Тестирование и отладка

Тест-кейсы для Prolog-запросов

№	Запрос	Ожидаемый ответ
T1	?- player(ivan).	true.
T2	?- char(torvald).	true.
T3	?- playing(P, C), char_info(C, mage).	P = artem, C = illidan.
T4	?- can_tank(C) ; can_inspire(C).	C = senshi ; C = torvald ; C = illidan ; C = vuga.
T5	?- will_conflict(torvald, Who).	Who = shmig ; Who = buga ; Who = vuga.
T6	?- party_tension(P1, P2).	P1 = ivan, P2 = oleg ; … (6 решений)
T7	?- age(illidan, Age, 2026).	Age = 1026.
T8	?- can_suppressed(shmig, torvald).	true.
T9	?- birth_info(senshi, 1830, ada).	true.
T10	?- knows_history(torvald, ada).	true.

Тест-кейсы для DL Query (Protégé)

№	DL Query	Ожидаемый результат
D1	Player	{ivan, oleg, artem, vanya}
D2	Character and hasClass value bard	{torvald}
D3	Character and bornRace value elf	{illidan}
D4	Character and birthPlace value ada	{torvald, shmig, senshi}
D5	Character and hasClass some MagicClass	{illidan, vuga}

Тесты DSS

№	Сценарий	Вход	Ожидаемое поведение
H1	Happy path — возраст	Сколько лет Иллидану?	Выводится Иллидан: примерно 1026 лет.
H2	Happy path — конфликт	Могут ли конфликтовать	Да, Сенши и Шмиг могут

		Сенши и Шмиг?	конфликтовать.
H3	Happy path — рождение	Родился ли Торвальд в Аде в 1899 году?	Да, Торвальд родился в Аде в 1899 году.
N1	Негативный — персонаж не родился	Сколько лет Буге? при YEAR=1800	Буга: ещё не родился.
N2	Негативный — больше 2 персонажей	могут ли конфликтовать Сенши, Шмыг и Торвальд?	В вопросе найдено больше двух персонажей. Уточните пару.
N3	Негативный — нераспознанный intent	Погода в Аде	Не удалось определить тип вопроса.

Оценка и интерпретация

Сравнение реализаций: Prolog vs OWL/Protégé

Критерий	**Prolog**	**OWL / Protégé**
Выражение правил вывода	Удобно: will_conflict/2, can_suppressed/2, age/3 записываются компактными клаузами с дизъюнкцией, отрицанием и арифметикой.	Правила в чистом OWL выразить нельзя; потребовались бы SWRL, которые в данной среде не подключились.
Структурирование предметной области	Неявное — через предикаты.	Явное: классы Character, Enemy ⊑ Character, MagicClass ⊑ Class, функциональные свойства hasClass, bornRace.
Расширяемость	Добавление нового правила — дописать условия.	Добавление класса/свойства — через интерфейс Protégé; автоматическая проверка непротиворечивости reasoner'ом.
Выполнение запросов	Быстрый логический вывод, несколько решений через backtracking.	DL Query — декларативные выборки по ограничениям; SPARQL в данной работе не использовался.
Прозрачность	Текст .pl читается напрямую.	Граф онтологии нагляден, но требует визуализатора.

Вывод: Prolog удобнее для задач, состоящих из правил, (конфликты, возраст, подавление), OWL — для **структурного** описания домена и проверки согласованности. В связке они покрывают оба аспекта DSS: логический вывод + онтологическая схема.

Идеи развития

1. **Новые классы и правила**: добавить сущности Item, Skill, правило can_use_spell, can_use_skill.

2. **Улучшение диалога**: при пустом результате задавать уточняющий вопрос («Уточните расу или класс?»).

3. **Подключение хранилища**: перенести онтологию в RDF-хранилище (GraphDB / Oxigraph) и выполнять полноценные SPARQL-запросы.

4. **SWRL-правила**: мигрировать часть Prolog-правил в SWRL для единого онтологического контура.

5. **Расширение парсера**: поддержка составных вопросов, контекста диалога, отрицаний («Кто НЕ может танковать?»).

Заключение

В ходе модуля была разработана база знаний по игровой предметной области на Prolog и создана соответствующая OWL -модель в Protégé. Prolog-часть содержит достаточный набор одноместных и многоместных фактов, а также семь правил, использующих конъюнкцию, дизъюнкцию, отрицание неравенства и арифметический расчёт. Это позволяет демонстрировать как прямой поиск фактов, так и получение новых сведений через правила.

В онтологии выделены основные классы, индивиды, объектные и дататип-свойства, а часть отношений сделана функциональной. Плагины Protégé подключить не удалось, поэтому SPARQL и SWRL не были использованы.

В ходе ЛР2 на основе базы знаний из ЛР1 построена система поддержки принятия решений для игровой предметной области. Реализованный конвейер «строка → слоты → Prolog-запрос → ответ» корректно обрабатывает вопросы о конфликтах персонажей, напряжении между игроками, возрасте, рождении, способностях и поиске по расам/классам.

Практическая польза подхода — в прозрачности правил (каждый ответ можно проследить до конкретного предиката или аксиомы), проверяемости (тест-кейсы для Prolog и DL Query) и повторном использовании формализованных знаний: ту же онтологию можно подключить к другому DSS-агенту, а Prolog-правила — переиспользовать в иных сценариях логического вывода.

Приложение

Ссылки на исходные файлы, упоминаемые в отчете: Гитхаб репозиторий